

Discussion 11

Note: Your TA will probably not cover all the problems on this worksheet. The discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, discussions, and homework.

This Week's Cool AI Demo/Video:

Anthropic credit research: <https://www.youtube.com/watch?v=Y6wiWlcH5jM>

Why aren't LLMs deterministic with temperature 0?:

<https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>

1 Positional Encoding

(a) Why do we need positional encoding? Describe a situation where word-order information is necessary for the task performed.

(b) What is *relative* positional encoding? How is it different from *absolute* positional encoding?

- (c) **Rotary positional embeddings (RoPE)** are a way of encoding token positions by rotating pairs of embedding dimensions by an angle that grows with the position index, so that inner products between tokens automatically depend only on their relative positions. They've become a de-facto default in many modern LLMs.

Consider a sequence of N two-dimensional vectors

$$x_n := (x_n^{(1)}, x_n^{(2)}), \quad n = 1, 2, \dots, N.$$

For a parameter $\theta \in \mathbb{R}$, the RoPE encoding at position n is

$$\text{RoPE}(x_n, n) = R(n\theta)x_n, \quad \text{where} \quad R(\alpha) := \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}.$$

Show that the dot product between two RoPE-encoded vectors depends only on their *relative* positions. In particular, prove that for any integers m , n , and k , the following holds:

$$\text{RoPE}(x, m)^\top \text{RoPE}(y, n) = \text{RoPE}(x, m+k)^\top \text{RoPE}(y, n+k).$$

2 Matching Similarity Matrices to Attention Heatmaps

Recall that in self-attention mechanisms, we compute the key-query similarity matrix QK^T , scale the matrix by $\frac{1}{\sqrt{D_k}}$, and then apply the softmax function row-wise to obtain attention scores. Understanding how pre-softmax similarity scores translate to post-softmax attention weights is crucial for interpreting attention patterns. Below are four 4×4 pre-softmax similarity matrices and four corresponding post-softmax attention heatmaps. Your task is to match each pre-softmax matrix to its corresponding heatmap and explain the transformation.

Pre-softmax matrices:

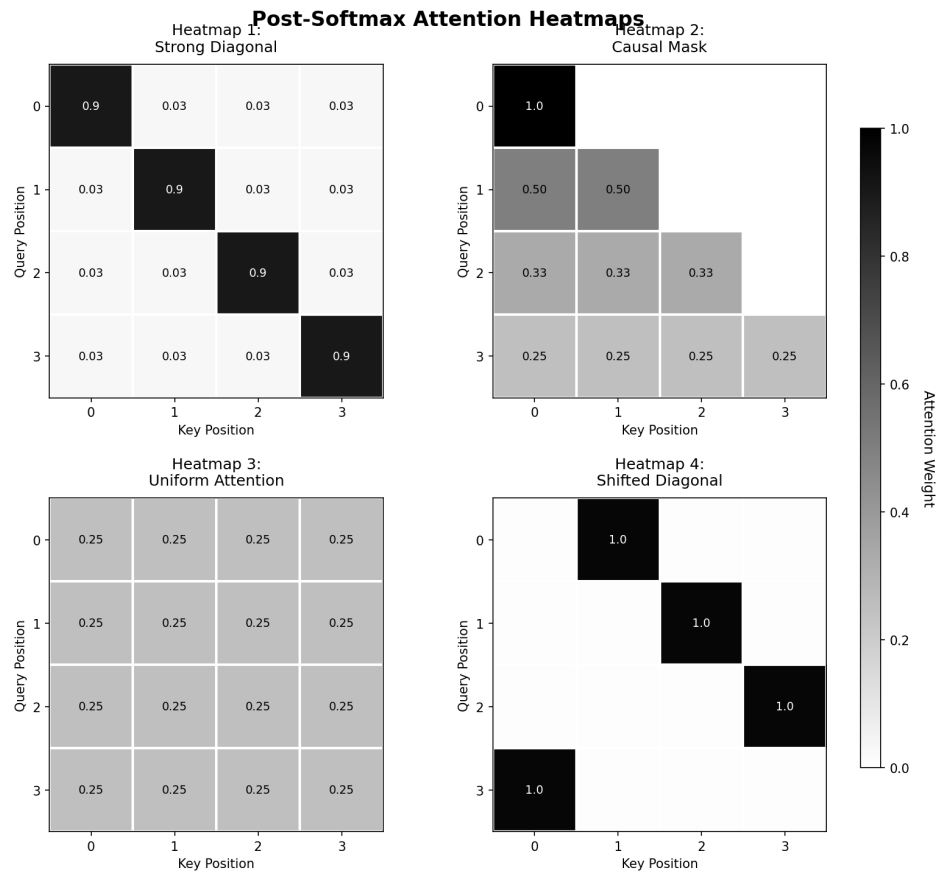
$$\text{Matrix A: } \begin{bmatrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{bmatrix}$$

$$\text{Matrix B: } \begin{bmatrix} 10 & 1 & 1 & 1 \\ 1 & 10 & 1 & 1 \\ 1 & 1 & 10 & 1 \\ 1 & 1 & 1 & 10 \end{bmatrix}$$

$$\text{Matrix C: } \begin{bmatrix} 5 & -\infty & -\infty & -\infty \\ 5 & 5 & -\infty & -\infty \\ 5 & 5 & 5 & -\infty \\ 5 & 5 & 5 & 5 \end{bmatrix}$$

$$\text{Matrix D: } \begin{bmatrix} -\infty & 5 & -\infty & -\infty \\ -\infty & -\infty & 5 & -\infty \\ -\infty & -\infty & -\infty & 5 \\ 5 & -\infty & -\infty & -\infty \end{bmatrix}$$

Post-softmax heat maps (darker = higher attention weight):



- (a) Match each pre-softmax matrix (A-D) to its corresponding heatmap (1-4). Provide brief reasoning for each match.
- (b) Explain why Matrix C represents a causal attention mask. In what type of model would you typically see this pattern?
- (c) Consider a temperature-scaled softmax: $\text{softmax}(x/T)$ where T is temperature. How would changing T affect the attention distribution for Matrix B? What happens as $T \rightarrow 0$ and $T \rightarrow \infty$?

3 KV Caching in Autoregressive Generation

In autoregressive language models like GPT, we generate tokens one at a time, conditioning on all previously generated tokens. This problem explores how KV caching dramatically improves the runtime of this process.

The Naive Approach:

Consider generating a sequence of length N with a transformer that has:

- Model (hidden) dimension, D_{model}
- Number of attention heads, H
- Number of attention layers, L

Without caching, at iteration n when generating token y_n , we compute attention as:

$$\text{Attention}(Q_n, K_{1:n}, V_{1:n}) = \text{SoftMax}_{\text{row}} \left(\frac{Q_n K_{1:n}^T}{\sqrt{D_{model}/H}} \right) V_{1:n}$$

where $K_{1:n}$ and $V_{1:n}$ are the keys and values for all tokens from position 1 to n . Notice that we recompute K_i and V_i for all $i < n$ at every step, even though these values don't change.

The KV Cache Solution: We store the computed K and V matrices from previous iterations and reuse them, only computing the new K_n and V_n for the current token.

- (a) **Without KV caching**, how many times do we compute the key and value projections for the **first** token y_1 throughout the entire generation of a sequence of length N ?

- (b) For a single layer, what is the total number of matrix multiplications needed to compute **all** key projections $K_{1:N}$ throughout the entire generation process **without KV caching**? (Assume we compute each K_i individually at each iteration, i.e. $K_i = y_i W_k$, instead of stacking tokens i up to n into an $n \times D_{model}$ matrix to compute $K = XW_k$ at each iteration.) Express your answer in terms of N .

- (c) **With KV caching**, how many matrix multiplications are needed to compute all key projections throughout the **entire** generation?
- (d) Briefly explain why KV caching is particularly important for applications like chatbots or code assistants that involve multiple rounds of interaction.